Course Name:	**Competitive Programming Lab**	**Semester:**	**IV**
Date of Performance:	**13/04/2026**	**DIV/ Batch No:**	**B2**
Student Name:	**Ashwera Hasan**	**Roll No:**	**16010124107**

Experiment No: 7
Title: String Algorithms.

Aim and Objective of the Experiment:

To study and implement string algorithms for cycle (periodicity) detection and analyze their efficiency in competitive programming.

Objectives

- **To understand the concept of cycles / periodic patterns in strings**
- **To detect repeating substrings using efficient string algorithms**
- **To implement cycle detection using Prefix Function (KMP) and Z-Algorithm**
- **To analyze time complexity of string cycle detection methods**
- **To apply periodicity detection in competitive programming problems**

COs to be achieved: CO4

Apply matrices, probability, game theory , and string algorithms in competitive programming.

Theory:

1. Prefix Function (π-array) Graph

- **X-axis:** String index (i)
- **Y-axis:** Prefix function value π[i]

Significance: A repeating pattern forms a **regular rise** in π-values, indicating a cycle in the string.

2. Z-Function Graph

- **X-axis:** String index

- **Y-axis:** Z-value

Significance: High Z-values at regular intervals indicate **periodic cycles** in the string.

3. Time Complexity Graph

- **X-axis:** Length of string (n)
- **Y-axis:** Execution time
- **Nature:** Linear → **O(n)**

Significance: Demonstrates suitability for large inputs in competitive programming.

Algorithms Used

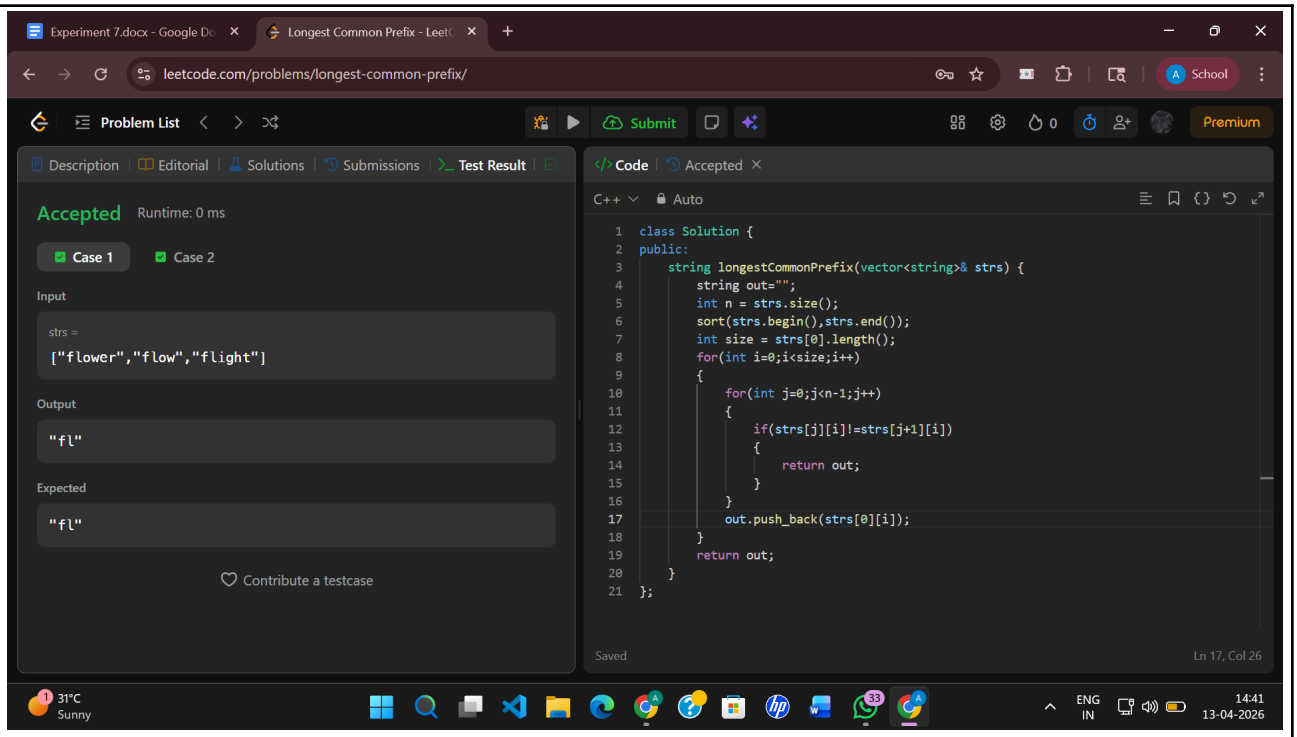
- **Prefix Function (KMP Algorithm)**
- **Z-Algorithm**
- **String Hashing**

Applications in Competitive Programming

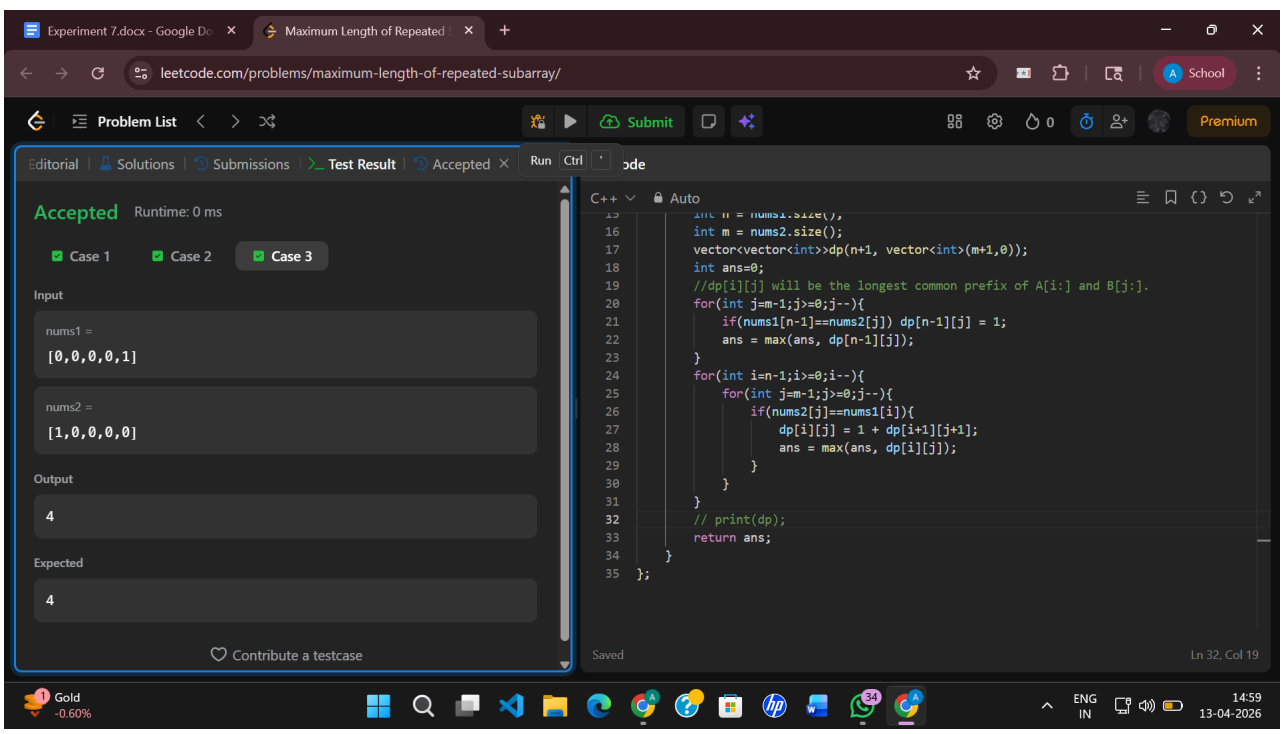
- Detecting **periodic strings**
- Finding smallest repeating unit of a string
- String compression problems
- DNA sequence analysis problems
- Pattern matching with repetitions
- Optimizing string matching problems

Problem Statements:
Easy : Longest common prefix : LC 14 – Longest Common Prefix Medium : Longest common substring : LC 718 – Maximum Length of Repeated Subarray Difficult : Rabin–Karp (rolling hash) : LC 1044 – Longest Duplicate Substring

Code :
Easy:


Experiment 7.docx - Google Do
Longest Common Prefix - LeetC
leetcode.com/problems/longest-common-prefix/
Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
Test Result
Code
Accepted
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
strs =
["flower","flow","flight"]
Output
"fl"
Expected
"fl"
Contribute a testcase
C++
Auto
class Solution {
public:
string longestCommonPrefix(vector<string>& strs) {
string out="";
int n = strs.size();
sort(strs.begin(),strs.end());
int size = strs[0].length();
for(int i=0;i<size;i++)
{
for(int j=0;j<n-1;j++)
{
if(strs[j][i]!=strs[j+1][i])
{
return out;
}
}
out.push_back(strs[0][i]);
}
return out;
}
};
Saved
Ln 17, Col 26
31°C
Sunny
ENG
IN
14:41
13-04-2026

Medium:


Experiment 7.docx - Google Do
Maximum Length of Repeated
leetcode.com/problems/maximum-length-of-repeated-subarray/
Problem List
Submit
Premium
Editorial
Solutions
Submissions
Test Result
Accepted
Run
Ctrl
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums1 =
[0,0,0,0,1]
nums2 =
[1,0,0,0,0]
Output
4
Expected
4
Contribute a testcase
C++
Auto
int m = nums2.size();
vector<vector<int>>dp(n+1, vector<int>(m+1,0));
int ans=0;
//dp[i][j] will be the longest common prefix of A[i:] and B[j:].
for(int j=m-1;j>=0;j--){
if(nums1[n-1]==nums2[j]) dp[n-1][j] = 1;
ans = max(ans, dp[n-1][j]);
}
for(int i=n-1;i>=0;i--){
for(int j=m-1;j>=0;j--){
if(nums2[j]==nums1[i]){
dp[i][j] = 1 + dp[i+1][j+1];
ans = max(ans, dp[i][j]);
}
}
}
// print(dp);
return ans;
}
};
Saved
Ln 32, Col 19
Gold
-0.60%
ENG
IN
14:59
13-04-2026

Hard:

Experiment 7.docx - Google Do | Longest Duplicate Substring - L

leetcode.com/problems/longest-duplicate-substring/

Problem List | Submit | Premium

Test Result | Accepted

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

s =
"banana"

Output

"ana"

Expected

"ana"

Contribute a testcas

Code

C++ | Auto

```cpp
public:
    string check(int size, string s) {
        if(size == 0) return "";

        const long long base = 31;
        const long long mod = 1e9 + 7;

        unordered_map<long long, vector<int>> seen;

        long long hash = 0, power = 1;

        for(int i = 0; i < size; i++){
            hash = (hash * base + (s[i] - 'a' + 1)) % mod;
            if(i < size - 1) power = (power * base) % mod;
        }

        seen[hash].push_back(0);

        for(int i = size; i < s.size(); i++){
            hash = (hash - (s[i - size] - 'a' + 1) * power % mod + mod) % mod;
            hash = (hash * base + (s[i] - 'a' + 1)) % mod;

            int start = i - size + 1;
```

Saved | Ln 9, Col 11

31°C Sunny | ENG IN | 15:44 13-04-2026

Code | Run Ctrl '

C++ | Auto

```cpp
            int start = i - size + 1;

            if(seen.count(hash)){
                for(int prev : seen[hash]){
                    if(s.substr(prev, size) == s.substr(start, size))
                        return s.substr(start, size);
                }
            }

            seen[hash].push_back(start);
        }

        return "";
    }
    string longestDupSubstring(string s) {
        int lo = 1, hi = s.size();
        string answer = "";

        while(lo <= hi){
            int mid = (lo + hi) / 2;
            string temp = check(mid, s);
```

Saved | Ln 9, Col 11

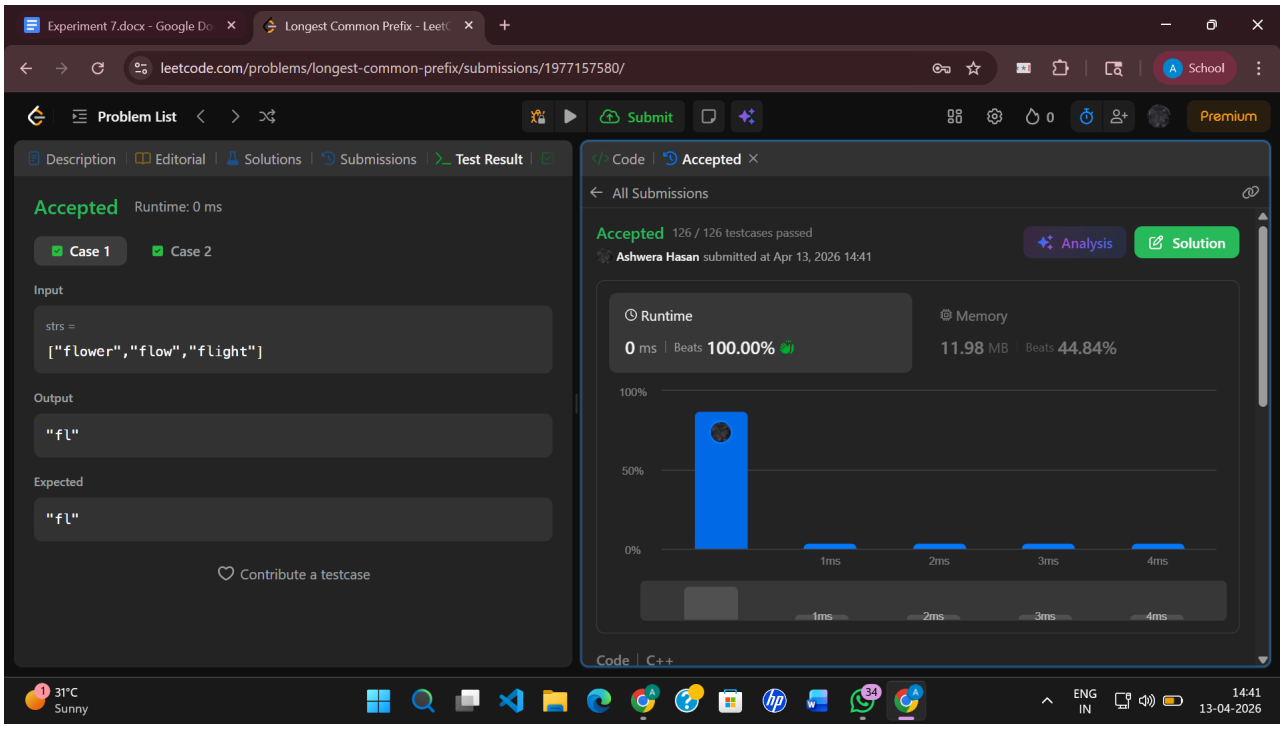
Code

C++ Auto

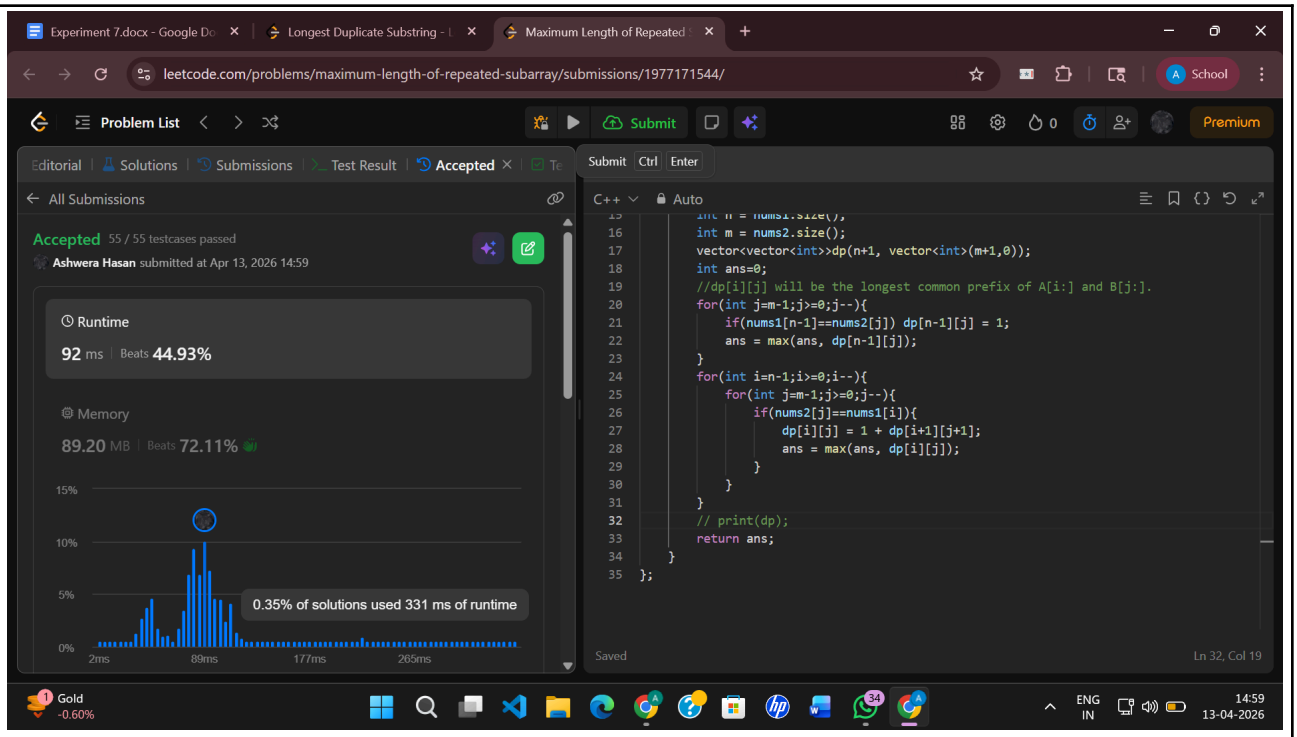
```cpp
    }
    string longestDupSubstring(string s) {
        int lo = 1, hi = s.size();
        string answer = "";

        while(lo <= hi){
            int mid = (lo + hi) / 2;
            string temp = check(mid, s);

            if(!temp.empty()){
                answer = temp;
                lo = mid + 1;
            } else {
                hi = mid - 1;
            }
        }
        return answer;
    }
};
```

Output:

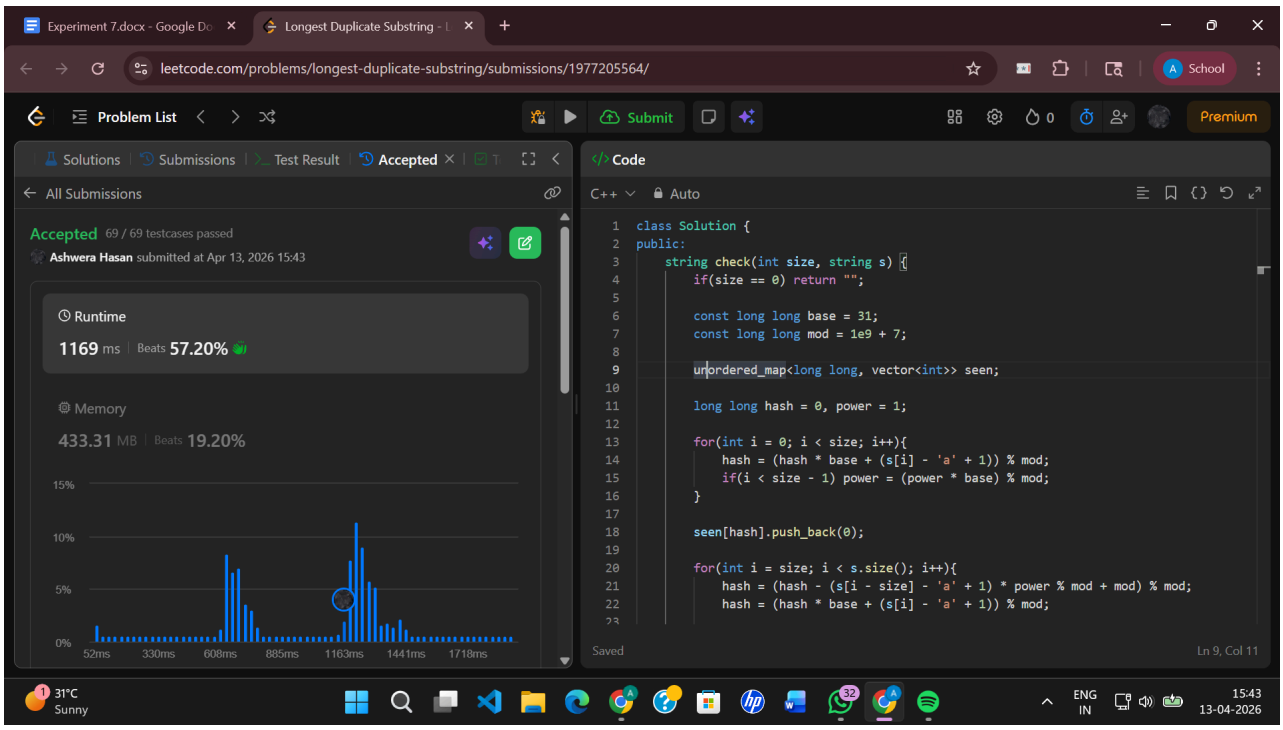
Easy:



Medium:



Hard:



Post Lab Subjective/Objective type Questions:

1. Why is the brute-force approach inefficient for finding duplicate substrings?
   The brute force approach is inefficient for finding duplicate substrings because then, the complexity will be O(n*m) since for each n substrings in s1, we need to check all m substrings in s2 for equality.
2. How does Binary Search help optimize substring length search?
   Binary search splits the searching space into half every time we run the check function. It avoids calculating the unnecessary half of the searching space. This reduces the time complexity to O(n log m)
3. Compare substring matching techniques: Brute Force vs Hashing vs DP.

   Substring matching techniques include brute force, hashing, and dynamic programming, each with different approaches and efficiency.

   Brute force method checks every possible position in the text and compares the pattern character by character. Its time complexity is O(n·m), making it simple but inefficient for large inputs.

   Hashing method, such as Rabin–Karp, uses a rolling hash to compare pattern and substring hashes instead of direct comparison. It has average time complexity O(n + m), but in case of hash collisions, it can degrade to O(n·m). It is faster in practice.

   Dynamic programming is used when patterns are complex, such as wildcard or regular expressions. It builds a table to store intermediate results. Its time and space complexity are both O(n·m), making it flexible but computationally expensive.

   Thus, brute force is easiest but slow, hashing is efficient on average, and dynamic programming is powerful but costly.

Conclusion:

In conclusion, the experiment helped understand and implement efficient string algorithms for detecting periodic patterns and repeating substrings. Techniques like the prefix function (KMP), Z-algorithm, and hashing provide optimal linear or near-linear performance, making them suitable for competitive programming. Compared to brute force methods, these approaches significantly reduce time complexity and handle large inputs efficiently. Overall, mastering these algorithms is essential for solving advanced string problems and optimizing pattern matching tasks.

Signature of faculty in-charge with Date: